29.2 技能结构

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~410k tokens，输出 ~7k tokens（本节）

上一节介绍了技能系统的整体架构。本节深入分析技能文件的具体规范——Front Matter 的每个字段意味着什么，内置技能目录的组织方式，以及几个代表性技能的设计思路。

29.2.1 SKILL.md 文件规范

文件结构

每个技能是一个目录，包含一个 `SKILL.md` 文件（或者技能也可以是一个单独的 `.md` 文件）。典型结构：

```
skills/
├── weather/
│   └── SKILL.md
├── github/
│   └── SKILL.md
├── discord/
│   ├── SKILL.md
│   └── （可能有辅助文件）
```

Front Matter 字段

`SKILL.md` 的 YAML Front Matter 定义了技能的元数据。以 `github` 技能为例：

```
---
name: github
description: "Interact with GitHub using the `gh` CLI."
metadata:
  {
    "openclaw":
      {
        "emoji": "🐙",
        "requires": { "bins": ["gh"] },
        "install":
          [
            { "id": "brew", "kind": "brew", "formula": "gh", "bins": ["gh"] },
            { "id": "apt", "kind": "apt", "package": "gh", "bins": ["gh"] },
          ],
      },
  }
---
```

完整字段说明：

字段	类型	必需	说明
`name`	string	是	技能唯一标识符
`description`	string	是	技能描述（≤100 字符，用于命令提示）
`homepage`	string	否	技能主页 URL
`user-invocable`	boolean	否	是否可被用户通过斜杠命令调用（默认 `true` ）
`disable-model-invocation`	boolean	否	是否禁止注入系统提示词（默认 `false` ）
`command-dispatch`	string	否	命令调度方式（ `tool` 表示确定性调度）
`command-tool`	string	否	确定性调度时调用的工具名
`metadata`	JSON5	否	OpenClaw 特定元数据（见下文）

OpenClaw 元数据（metadata.openclaw）

`metadata` 字段内嵌一个 JSON5 对象，其 `openclaw` 键包含 OpenClaw 特定的技能配置：

```typescript
// src/agents/skills/types.ts

export type OpenClawSkillMetadata = {
  always?: boolean;          // 无条件激活（跳过所有资格检查）
  skillKey?: string;         // 技能配置键（默认使用 name）
  primaryEnv?: string;       // 主要环境变量名（如 DISCORD_TOKEN）
  emoji?: string;            // 显示用 emoji
  homepage?: string;         // 主页 URL
  os?: string[];             // 支持的操作系统 ["darwin", "linux"]
  requires?: {
    bins?: string[];         // 必需的二进制文件（全部需要）
    anyBins?: string[];      // 至少需要一个的二进制文件
    env?: string[];          // 必需的环境变量
    config?: string[];       // 必需的配置路径（如 "browser.enabled"）
  };
  install?: SkillInstallSpec[];  // 安装规格（多种安装方式）
};
```

`requires` 是技能资格过滤的核心——它声明了技能运行所需的外部依赖。系统启动时检查这些依赖是否满足，不满足的技能自动隐藏。

安装规格（SkillInstallSpec）

`install` 数组定义了如何安装技能的外部依赖，支持五种安装器：

```typescript
export type SkillInstallSpec = {
  id?: string;               // 安装选项标识符
  kind: "brew" | "node" | "go" | "uv" | "download";
  label?: string;            // 用户可见的安装描述
  bins?: string[];           // 安装后应存在的二进制文件
  os?: string[];             // 此安装方式适用的操作系统
  formula?: string;          // brew: Homebrew formula 名
  package?: string;          // node/uv: 包名
  module?: string;           // go: 模块路径
  url?: string;              // download: 下载 URL
  archive?: string;          // download: 归档格式
  extract?: boolean;         // download: 是否解压
  targetDir?: string;        // download: 目标目录
};
```

同一个技能可以提供多种安装方式。例如 `github` 技能同时支持 `brew install gh` 和 `apt install gh` ——系统根据用户偏好和可用的包管理器选择最合适的方式。

Front Matter 解析

`parseFrontmatter` 从 Markdown 文件提取 YAML 头部， `resolveOpenClawMetadata` 从中提取 OpenClaw 元数据：

```
// src/agents/skills/frontmatter.ts（简化）

export function resolveOpenClawMetadata(
  frontmatter: ParsedSkillFrontmatter,
): OpenClawSkillMetadata | undefined {
  const raw = frontmatter["metadata"];
  if (!raw) return undefined;

  const parsed = JSON5.parse(raw);
  // 尝试从 "openclaw" 键（及兼容键）中提取元数据
  const metadataRaw = parsed["openclaw"] ?? parsed["clawdbot"] ?? parsed["claude-code"]

  return {
    always: metadataRaw.always,
    emoji: metadataRaw.emoji,
    primaryEnv: metadataRaw.primaryEnv,
    requires: metadataRaw.requires ? {
      bins: normalizeStringList(metadataRaw.requires.bins),
      anyBins: normalizeStringList(metadataRaw.requires.anyBins),
      env: normalizeStringList(metadataRaw.requires.env),
      config: normalizeStringList(metadataRaw.requires.config),
    } : undefined,
    install: metadataRaw.install?.map(parseInstallSpec).filter(Boolean),
  };
}
```

注意兼容性处理——代码尝试从多个可能的键名中读取元数据（`openclaw` 、 `clawdbot` 、 `claude-code` ），这是 OpenClaw 从其他项目演化而来时保留的兼容层。

29.2.2 内置技能目录分析

52 个内置技能

OpenClaw 的 `skills/` 目录包含 52 个内置技能。按功能领域分类：

开发与代码

- `github` — GitHub CLI 操作（PR、Issue、CI）
- `coding-agent` — 子 Agent 编排（Codex、Claude Code、Pi）
- `tmux` — 终端复用器操作
- `session-logs` — 会话日志查看

通信与社交

- `discord` — Discord 消息读写
- `slack` — Slack 频道操作
- `imsg` / `bluebubbles` — iMessage 集成
- `himalaya` — 电子邮件客户端

笔记与知识管理

- `obsidian` — Obsidian 笔记库操作
- `notion` — Notion 页面管理
- `bear-notes` — Bear 笔记
- `apple-notes` / `apple-reminders` — macOS 原生应用

智能家居

- `openhue` — Philips Hue 灯光控制
- `sonoscli` — Sonos 音响系统

AI 与模型

- `gemini` — Google Gemini API
- `oracle` — 高质量推理模型咨询
- `openai-image-gen` — DALL-E 图像生成
- `openai-whisper` / `openai-whisper-api` — 语音转文字

媒体与创作

- `spotify-player` — Spotify 播放控制

- `video-frames` — 视频帧提取
- `camsnap` — 摄像头截图
- `canvas` — 画布绘图

实用工具

- `weather` — 天气查询
- `summarize` — 文本摘要
- `healthcheck` — 服务健康检查
- `model-usage` — 模型使用统计
- `blogwatcher` — 博客监控

生态系统

- `clawhub` — 技能注册中心 CLI
- `skill-creator` — 技能创作向导

技能复杂度光谱

内置技能从极简到复杂覆盖了完整光谱：

级别	技能	SKILL.md 行数	特点
极简	`weather`	~55 行	单个 `curl` 命令，无 API Key
简单	`github`	~100 行	单个 CLI 工具的多命令用法
中等	`discord`	~200 行	需要 API Token + 多端点
复杂	`coding-agent`	~300 行	编排外部 Agent 进程，PTY 交互

29.2.3 代表性技能解读

weather：零依赖的简洁范例

`weather` 是最简单的技能之一——它展示了"技能即文档"的纯粹形态：

```
metadata: { "openclaw": { "emoji": "🌤", "requires": { "bins": ["curl"] } } }
```

唯一的依赖是 `curl` （几乎所有类 Unix 系统都有）。正文提供了两个天气服务的用法（wttr.in 和 Open-Meteo），包含具体的 `curl` 命令和格式化参数。Agent 读取这些内容后，用户问"今天北京天气如何？"时，Agent 就知道执行 `curl -s "wttr.in/Beijing?format=3"` 。

github：CLI 集成的标准模式

`github` 技能展示了如何集成一个成熟的 CLI 工具：

```
metadata:
  openclaw:
    emoji: "🐙"
    requires: { bins: ["gh"] }
    install:
      - { id: "brew", kind: "brew", formula: "gh", bins: ["gh"] }
      - { id: "apt", kind: "apt", package: "gh", bins: ["gh"] }
```

关键设计：

1. **多安装方式**：macOS 用户可以 `brew install gh` ，Linux 用户可以用 `apt`
2. **结构化正文**：按功能分组（PR、Issue、CI），每组提供最常用的命令
3. **注意事项**：提醒 Agent 在非 Git 目录时使用 `--repo owner/repo`

coding-agent：复杂交互场景

`coding-agent` 是最复杂的内置技能之一——它指导 Agent 如何启动和管理其他编码 Agent（如 Codex、Claude Code）：

```
metadata:
  openclaw:
    emoji: "🧩"
    requires: { anyBins: ["claude", "codex", "opencode", "pi"] }
```

注意 `anyBins` 而非 `bins` ——只要安装了其中一个编码 Agent CLI 就满足条件。正文中详细说明了：

- PTY 模式的必要性（编码 Agent 是交互式终端应用）
- 后台运行和会话管理
- 各工具参数（ `command` ， `pty` ， `workdir` ， `background` ）
- Process Tool 的操作（ `poll` ， `log` ， `write` ， `kill` ）

clawhub：生态自举

`clawhub` 技能体现了一种有趣的"自举"设计——它是一个技能，功能是管理其他技能：

```yaml
metadata:
  openclaw:
    requires: { bins: ["clawhub"] }
    install:
      - { id: "node", kind: "node", package: "clawhub", bins: ["clawhub"] }
```

Agent 通过 `clawhub` 技能学会如何搜索、安装、更新技能。当用户说"我需要一个能操作 Trello 的技能"时，Agent 会执行 `clawhub search "trello"`，找到并安装相应技能。

29.2.4 技能调用策略

每个技能有两个调用控制开关，通过 Front Matter 设置：

```typescript
// src/agents/skills/types.ts

export type SkillInvocationPolicy = {
  userInvocable: boolean;          // 是否可通过 /command 调用
  disableModelInvocation: boolean; // 是否禁止注入系统提示词
};
```

四种组合：

userInvocable	disableModelInvocation	效果
`true`	`false`	**默认**：Agent 自动感知 + 用户可手动调用
`true`	`true`	**仅用户调用**：不注入提示词，只能通过 /command 触发
`false`	`false`	**仅自动**：Agent 自动感知，但无斜杠命令
`false`	`true`	**完全禁用**：既不注入提示词，也无命令

`disableModelInvocation` 适用于需要用户明确意图才应触发的操作（如发送消息到 Slack），避免 Agent 自作主张。

本节小结

1. **SKILL.md 文件规范**包含 YAML Front Matter（元数据）和 Markdown 正文（领域知识），通过 `metadata.openclaw` 声明依赖、安装方式和特殊行为。
2. **五种安装器**（brew/node/go/uv/download）覆盖了主流包管理器和直接下载场景，同一技能可提供多种安装选项。
3. **52 个内置技能**从极简（weather，55 行）到复杂（coding-agent，300+ 行）覆盖完整光谱，涵盖开发、通信、智能家居、媒体等领域。
4. **调用策略**通过 `userInvocable` 和 `disableModelInvocation` 两个布尔值控制技能的触发方式，平衡自动化和用户控制。
5. **自举设计**： `clawhub` 技能是一个管理技能的技能， `skill-creator` 技能是一个创建技能的技能——技能系统用自身的机制来扩展自身。